1. Klasy

Deklaracja klasy nie tworzy obiektów

Dostęp do składników: nazwaobiektu.nazwaZmiennej, nazwaobiektu.nazwaFunkcji()

Konstruktor: uruchamiany podczas tworzenia obiektu, jak nie zrobimy swojego to jest domyślny, zwykle nim ustawiamy wartości domyślne składników, przy dziedziczeniu możemy go wykorzystać jak funkcję w celu ustawienia nim wartości zmiennych klasy z której dziedziczymy. Można przeładowywać konstruktor:

```
Address::Address(int actualNumber):      //constructor name same as class name
    houseNumber(actualNumber)            //initialization list, nie obowiazkowe
    {...}                                //constructor body

Address::Address(string strName):        //constructor name same as class name
    street(strName)                      //initialization list, nie obowiazkowe
    {...}                                //constructor body
```

Destruktor: wywoływany w chwili likwidacji obiektu

~nazwaKlasy(){..kod..}

Przykład:

zakładając stworzenie 2 obiektów (np. A i B) najpierw wywoła się konstruktor dla A, potem konstruktor dla B, następnie destruktor dla B i na końcu destruktor dla A

Przyjaźń - wewnątrz klasy wpisujemy:

friend class ZaprzyjaznionaKlasa; //friend <deklaracja klasy>

friend void ZaprzyjaznionaFunkcja();

Zaprzyjaźnione mają dostęp do prywatnych i chronionych składników klasy

Przyjaciel mojego przyjaciela NIE JEST moim przyjacielem.

Przeładowanie operatorów:

```
45
46 Complex operator+(Complex a1, Complex a2)
47 {
48     return Complex(a1.re+a2.re, a1.im+a2.im);
49 }
50
```

Pozwala dodać nową opcję, ale nie pozwala zmienić priorytetów operatorów, reguł łączności i liczby operatorów. Trzeba pamiętać aby zaprzyjaźnić operator, jeśli jest poza klasą!

Reguła: jeżeli operator zmienia zawartość składników klasy to przeładowujemy w klasie. Przykładem są: =,++,+=,*=

Operatory przeładowane w klasie mają jeden argument mniej.

```
65 Complex operator+(Complex);
66 };
67
68 Complex Complex::operator+(Complex a1)
69 {
70     return Complex(re+a1.re, im+a1.im);
71 }
72
73 double second;
74 Complex result=second+argument;
75 Complex result=second.operator(argument); //
   WRONG!!!!!
```

Dziedziczenie - no klasyk jak przykładowo okrąg dziedziczy z punktu
kod: *class Circle: public Point{...};*

Klasa_pochodna:**private** klasa_bazowa:

- Private - niedostępne z klasy pochodnej
- Protected, public - jest przenoszone do części prywatnej klasy pochodnej

Klasa_pochodna:**protected** klasa_bazowa:

- Private - niedostępne z klasy pochodnej
- Protected, public - jest przenoszone do części protected klasy pochodnej

Klasa_pochodna:**public** klasa_bazowa:

- Private - niedostępne z klasy pochodnej
- Protected, public - pozostają tam gdzie były

Podczas tworzenie obiektu konstruktory klas bazowych wywoływane od klasy najstarszej w hierarchii.
Tak samo nazwane składniki wymagają operatora zakresu ::
Przyjaźń nie jest dziedziczna

2. Wzorce klas

Umożliwiają definiowanie np. koła ma bazie klasy kształt etc.

np.
```
template <class Item> class Object {
        Item costam1;
        Item costam2;
        //tu konstruktor wrzucajacy do zmiennych wyzej
};
Object <float> paraFloatow(6.3, 1.6);
```

3. Wskaźniki

```
int *Pointer;                // zmienna która będzie przechowywać adres
int number = 3;              // no int zwykły
Pointer = &number;           // & przekazuje adres zmiennej, czyli teraz Pointer ma adres
int value = *Pointer;        // odczyt zawartości wskazanej zmiennej
                             // innymi słowy: weź zobacz na co pokazuje Pointer i przypisz
                             // tę wartość zmiennej value, czyli teraz value = 3
```

Dynamiczne tworzenie obiektów - za pomocą operatorów new i delete
memory leak jeśli więcej new od delete

Obiekty takie nie mają nazw, dostęp tylko przez wskaźnik.

przykłady:

*complex *number = new complex(2,3);*

nazwa tablicy adresem jej pierwszego elementu;

**(Vector+5)* oznacza przesunięcie o 5 słów binarnych (o 5 komórek)

```
double *Vector = new Double[size];
Vector[5] = 33;                 // to samo co niżej
*(Vector+5)=33;                 // to samo co wyżej
Vector[0]=15;                   // to co niżej
*Vector=15;                     // to co wyżej
delete [] Vector;               // usuwa tablicę

Pointer = array;                // Pointer = &array[0];
for(i=0;i<4;i++) *(Pointer++)=-1    // tu zmiana wartości pierwszych 4 z tablicy na -1
```

Tablica dwuwymiarowa

4. **Funkcje wirtualne i polimorfizm // s18 u rudego**
5. **Pojemniki**
6. **Listy jednokierunkowe**
7. **jakieś inne listy**
8. **Kolejka fifo**
9. **Kolejka lifo - stos**
10. **Notacja polska i ONP**

Zapis bez nawiasów i tyle, co tu gadać

Odwrotna Notacja Polska - znak działania po operandach

przykład rozwiązywania: 1 1 + 5 3 - 5 * 2 / ^ //weź 2 liczby, znak po nich i licz

2 2 5 * 2 / ^

2 10 2 / ^

2 5 ^

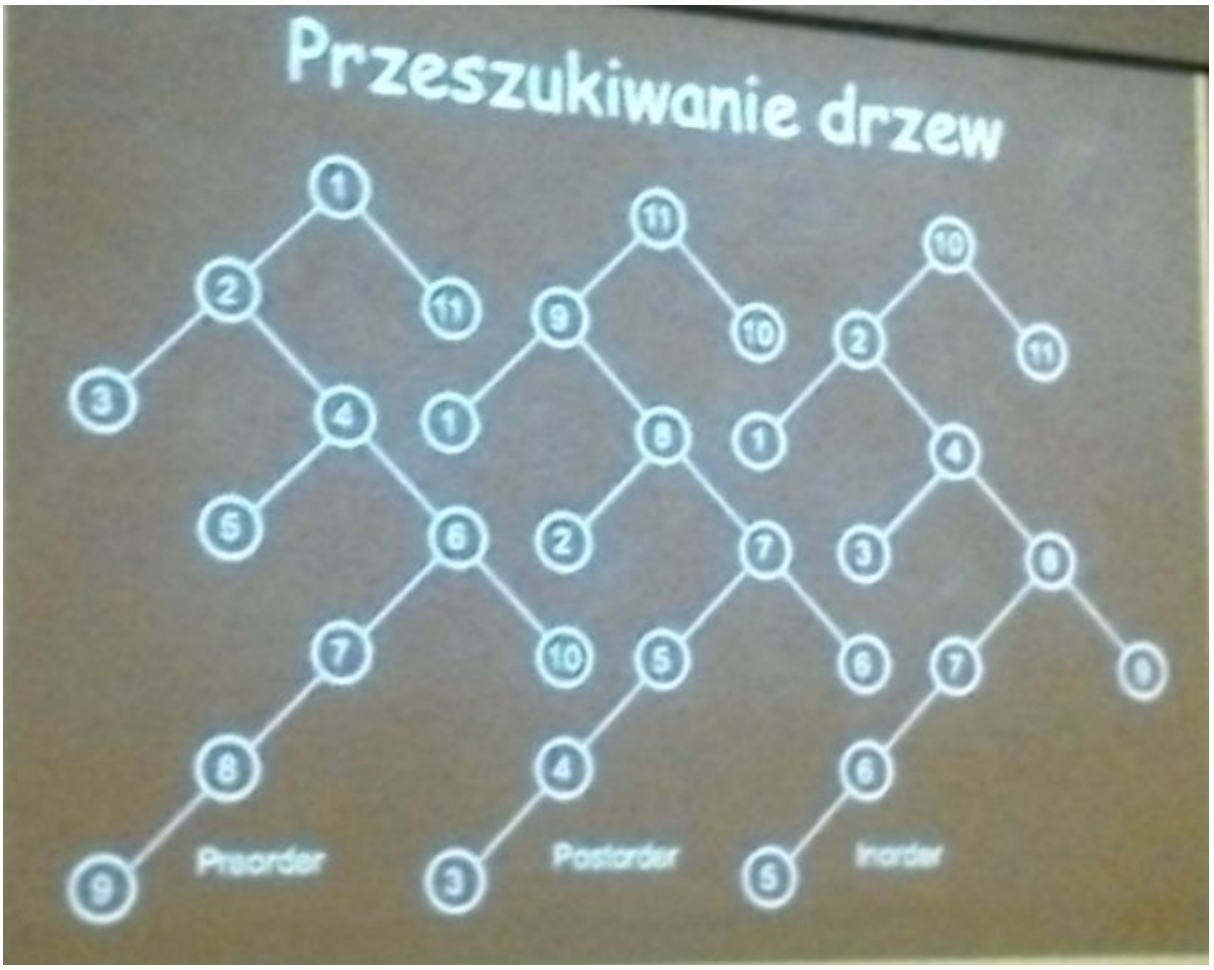
32

11. **Drzewa binarne**
12. **Metody przeszukiwania drzewa**

Metody przeszukiwania drzew

- porządek poprzedzający (ang. preorder)
  - zbadaj wierzchołek bieżący
  - zbadaj lewe poddrzewo
  - zbadaj prawe poddrzewo
- porządek następujący (postorder)
  - zbadaj lewe poddrzewo

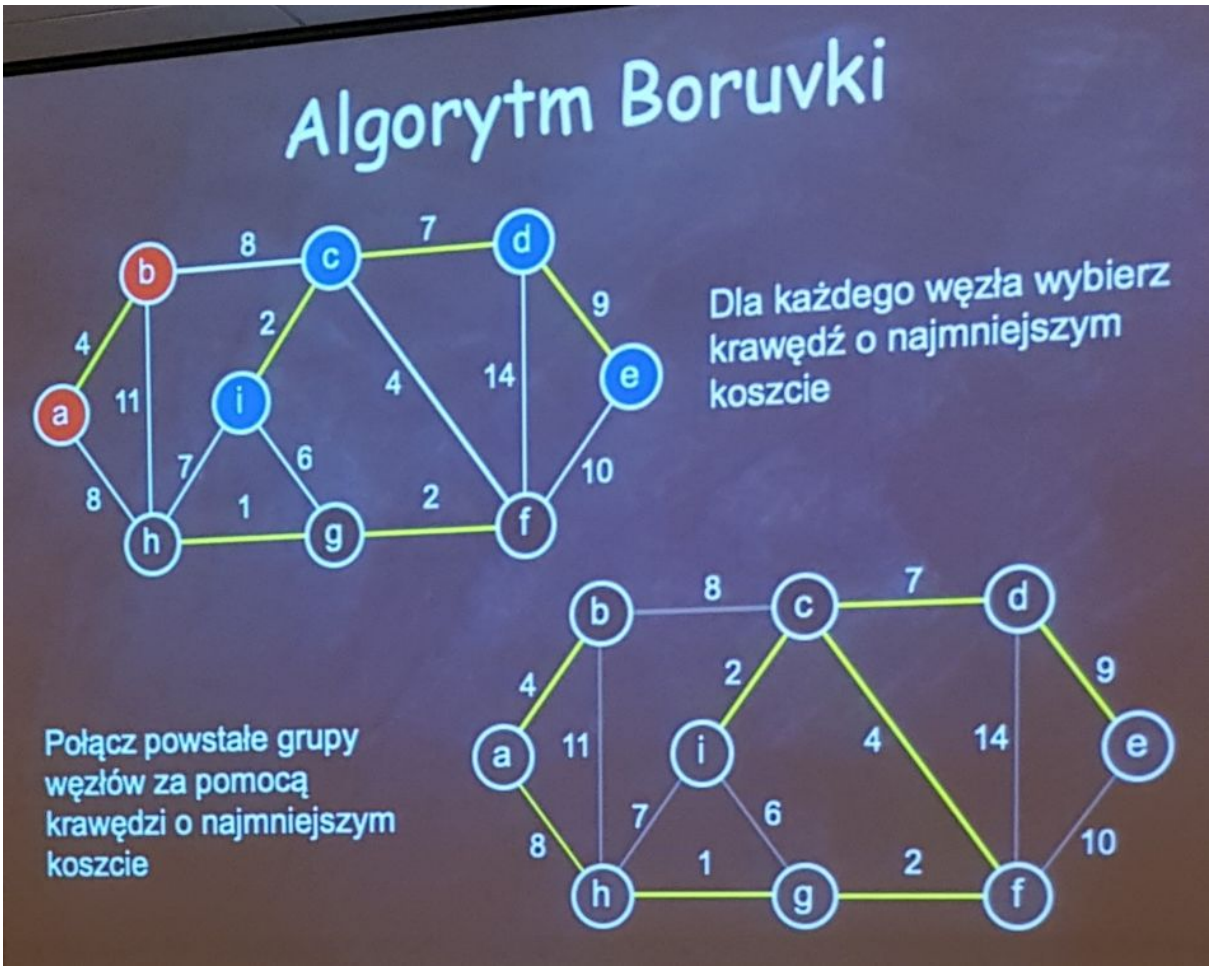
        - zbadaj prawe poddrzewo
        - zbadaj wierzchołek bieżący
- porządek wewnętrzny (inorder)
    - zbadaj lewe poddrzewo
    - zbadaj wierzchołek bieżący
    - zbadaj prawe poddrzewo



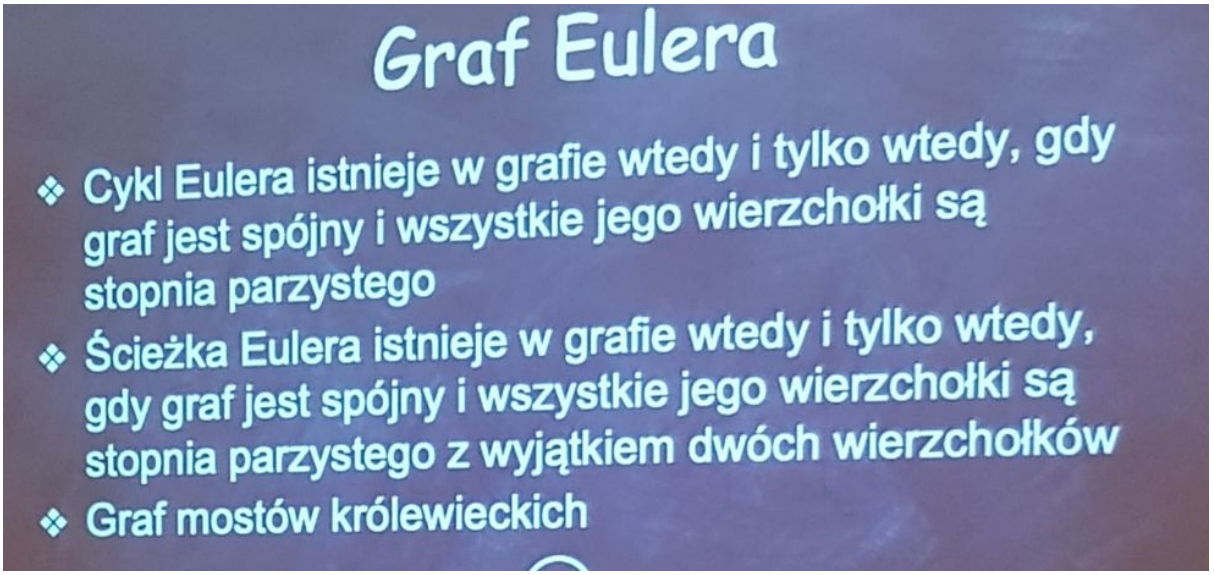
//sory za jakość ale to rudego zdjęcie, no to czego się spodziewać

13. **Drzewa: https://www.youtube.com/watch?v=6SLizJBvEg8**
14. **Drzewa 2: https://www.youtube.com/watch?v=kPb8LIlcYv0**
15. **Drzewa 3: https://www.youtube.com/watch?v=326CFNVrsQ0**

16. **Przeszukiwanie grafu wszesz w głąb : https://www.youtube.com/watch?v=CE4cyDpY2vc**
17. **Algorytm Prima : https://www.youtube.com/watch?v=fjMSreoCb7c**
18. **Algorytm Kruskala : https://www.youtube.com/watch?v=UnEGd4m8-uo**
19. **Algorytm Boruvki:**



20. Cykl Eulera: https://eduinf.waw.pl/inf/alg/001_search/0134.php



Cykl Eulera (ang. Eulerian circuit, Eulerian cycle lub Eulerian tour) jest ścieżką, która przechodzi dokładnie jeden raz przez każdą krawędź grafu i kończy się w wierzchołku, od którego się rozpoczęła.

Ścieżka Eulera (ang. Eulerian path, Eulerian trail lub Eulerian walk) to ścieżka, która przechodzi dokładnie jeden raz przez każdą krawędź grafu, lecz kończy się w innym wierzchołku niż w startowym.

21. Cykl Hamiltona: https://www.youtube.com/watch?v=6QFSkhcHLiAN